Git Revert vs Git Reset

Back to: Microservices using ASP.NET Core Web API Tutorials

Git Revert vs Git Reset

When we work on a project, mistakes happen very often. Sometimes we commit the wrong code. Sometimes we delete something important. Sometimes we test a change, commit it, and later realize it should not go to main. In such situations, Git gives us ways to undo changes.

Whenever you want to undo something in Git, ask this first:

- Has this commit already been pushed to GitHub?
- Are other developers already using this commit?
- Is this a shared branch like main?
- Or is this only my local branch and my local commit?

This one decision changes everything.

Practical Rule

- If the commit is already pushed and shared, use **Revert**
- If the commit is still local and private, **Reset** may be used

That is the simplest and safest practical understanding.

Real-Time Scenario Setup

Suppose you are working on an ASP.NET Core Web API project. Your commit history looks like this:

- Commit 1: Initial Product API setup
- Commit 2: Add CategoriesController
- Commit 3: Add validation in ProductsController
- Commit 4: Accidentally remove price validation

Now, later, you discover that **Commit 4 is wrong**. At this point, the next action depends on whether Commit 4 has already been pushed or not.

Situation 1: The Bad Commit Is Already Pushed to GitHub

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

This is the most common team scenario. Suppose you already pushed the bad commit to GitHub. Maybe your team has already pulled it. Maybe a pull request is already open. In this case, you should safely undo the bad commit by creating another commit that reverses it. That is exactly where **Git Revert** is used.

So, Git Revert says that the old commit was wrong. Let me create a new commit that cancels its effect. So, the bad commit remains in history, and a new commit is added to reverse it.

Practical Example of Git Revert

Suppose the history is:

- Initial commit
- Add CategoriesController
- Add Product Validation
- Remove Product Validation by mistake

Now you realize the last commit is wrong. Instead of deleting that commit from history, you run revert. Git creates a new commit like:

- Revert “Remove Product Validation by mistake”

Now the project comes back to the correct state, but the full history still shows:

- What mistake happened
- When it happened
- Who corrected it
- How it was corrected

This is excellent for real project tracking and team collaboration.

How to Revert a Commit Using Visual Studio

Now, let us make it practical in Visual Studio.

Step 1: Open the Project

Open your existing project in Visual Studio.

Step 2: Open Branch History

From the top menu:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Click **Git**
- Click **View Branch History**

Now Visual Studio will show the commit history of the current branch.

Step 3: Find the Commit to Undo

In the history window:

- Find the wrong commit
- Read the commit message carefully
- Make sure you are selecting the correct one

This is very important because revert will reverse the selected commit.

Step 4: Right-Click the Commit

Now right-click the bad commit.

Step 5: Click Revert

Select **Revert**. Visual Studio will automatically create a new commit that undoes the previous one.

Step 6: Push the Revert Commit

Now push the new commit to GitHub. That is the full safe workflow.

Situation 2: The Bad Commit Is Only on My Local Machine

Now, let us consider another scenario. Suppose you made a commit locally but have **not pushed it yet**. Maybe you are still cleaning your work. Maybe the commit message is wrong. Maybe you want to remove the last commit and re-commit it properly. In this case, you can use **Git Reset**.

So, Git Reset says, Move my branch back to an older point. Reset is mainly useful when:

- Work is still local
- Commit is not shared
- You want to clean up local history
- You want to remove or redo recent commits

Why Git Reset Can Be Dangerous

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Reset becomes dangerous when a developer uses it on a shared branch and then force-pushes the result. Because reset changes the branch history. That means:

- Commits may disappear from the branch
- Other developers may already have those commits
- Local and remote history may stop matching
- The team may get confused
- Merge and pull problems may happen

So, Reset is a local cleanup tool, not a team-safe undo tool.

Practical Use Case of Git Reset

Suppose you made 3 local commits:

- Add CategoriesController
- Add Product Validation
- Temporary test code

Now you realize the last commit is useless and should not exist. If it is still local and not pushed, you can use reset to move back before that commit. This is a very practical use of reset.

Soft Reset vs Hard Reset – Practical Explanation

Soft Reset – Undo Commit, Keep the Code

A **soft reset** removes the commit from commit history, but keeps the code changes available. It means:

- Commit is removed
- Code is still there
- Changes remain ready for recommit
- Useful when the commit message is wrong
- Useful when you want to combine commits
- Useful when you commit too early

Example

You committed:

- Added Product Validation

But later you realize:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- The commit message is poor
- One more file must be added
- You want to recommit properly

Now you can do a soft reset.

Result:

- The commit is undone
- The code remains with you
- You can make corrections
- Then commit again with a better message

This is very useful in local cleanup.

Hard Reset – Undo Commit and Remove the Code

A **hard reset** removes the commit and the changes from your working state. It means:

- Commit is removed
- Code changes are also discarded
- Useful only when you are fully sure
- Risky if used carelessly

Example

Suppose you committed some experimental test code, and now you want to completely throw it away. Then a hard reset may be used. But if you are not careful, a hard reset can throw away valuable work. That is why beginners should be very careful with a hard reset.

How to Reset a Commit in Visual Studio

Now, let us see the practical Visual Studio flow.

Step 1: Open the Project

Open your solution in Visual Studio.

Step 2: Open Branch History

From the top menu:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Click **Git**
- Click **View Branch History**

Step 3: Find the Commit

Locate the commit where you want to reset back to.

Step 4: Right-Click the Commit

Visual Studio gives reset options.

Depending on the version, you may see options similar to:

- **Reset > Keep Changes**
- **Reset > Delete Changes**

Step 5: Choose the Correct Reset Type

Reset > Keep Changes

This is the safer local cleanup option.

Practical meaning:

- Move the branch back
- Keep the code changes
- Let me recommit properly later

Reset > Delete Changes

This is the risky option.

Practical meaning:

- Move the branch back
- Remove the changes
- Discard that work completely

Step 6: Verify Carefully

After reset:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Check changed files
- Confirm code state
- Make sure you got the expected result

Never do a reset blindly.

Which Is Safer in Teams: Revert or Reset?

In teams, Revert is the default safe option. Reset is mostly for local cleanup before sharing.

Why Revert Is Safer

- It does not rewrite shared history
- It keeps team collaboration stable
- It works well on pushed commits
- It avoids force-push problems
- It makes history easier to understand

Why Reset Is Risky in Teams

- It changes history
- Shared commits may disappear from the branch
- Others may already be working on those commits
- Force push may be required
- Team members may face branch mismatch issues

Conclusion

git revert and git reset are both used to undo work, but they are not interchangeable. Revert is the safe, collaboration-friendly way to undo an earlier commit because it creates a new commit and preserves history. Reset is a history-moving command that is useful for local cleanup, but it can be dangerous if misused on shared branches.

So, always think before choosing the tool. Ask yourself one question first: **Is this commit already shared with others?** If the answer is yes, prefer to **revert**. If the answer is no and you are only cleaning your own local branch, then **resetting** may be fine.

DOT NET TUTORIALS

Dot Net Tutorials

About the Author: Pranaya Rout

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “Git Revert vs Git Reset”

DOT NET TUTORIALS
APRIL 6, 2026 AT 11:51 AM

🎥 Prefer Learning with a Real-Time Example?

If you want to clearly understand how Git and GitHub work together in real-world ASP.NET Core development, we’ve created a complete step-by-step video tutorial for you.

👉 Watch the full video here:
https://youtu.be/w9nG2DLrlsc

In this video, you will learn:
• Difference between Git and GitHub (very clearly explained)
• How local repository connects to GitHub
• Push and Pull workflow in real projects
• Feature branch and bugfix workflow
• Pull Requests (PR) with real-time example
• Code review, approval, and merge process

💡 This is highly recommended if you want to work in team-based projects or prepare for real-world development scenarios.

Don’t miss this — it will make Git & GitHub concepts very easy to understand

Reply

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Consent Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF